Machine Learning Assignment 2

Dry Bean Multiclass Classification

Student Name: _____
Student ID: _____
Course: M.Tech AIML
GitHub Repository: TO BE ADDED
Live Streamlit App: TO BE ADDED

Problem Statement

Predict seven dry-bean varieties from 16 image-derived numerical features using five mandatory classification algorithms.

Dataset Description

UCI Dry Bean Dataset, ID 602, approximately 13,611 instances, 16 features, target Class, seven categories.

Preprocessing

Duplicate review/removal, stratified 80:20 split, training-only median imputation and scaling through pipelines.

Models and Metrics

Logistic Regression, Decision Tree, KNN, Gaussian Naive Bayes, and SVM. Metrics: Accuracy, macro OVR AUC, macro Precision, macro Recall, macro F1, and MCC.

Model Comparison Table

Model	Accuracy	AUC	Precision	Recall	F1	MCC
Logistic Regression						
Decision Tree						
KNN						
Naive Bayes						
SVM						

Performance Observations and Overall Winner

Populate only after actual execution. Select the overall winner using multiple metrics, especially macro F1, MCC, and AUC, not accuracy alone.

BITS Virtual Lab Screenshot

INSERT ONE VERIFIED BITS VIRTUAL LAB SCREENSHOT HERE

README Content

Machine Learning Assignment 2: Dry Bean Classification

a. Problem Statement

Build and compare five multiclass classifiers that predict one of seven dry-bean varieties from 16 image-derived numerical features, then evaluate saved pipelines through Streamlit without retraining.

b. Dataset Description

UCI Dry Bean dataset, dataset ID 602: approximately 13,611 instances, 16 numerical predictors, target `Class`, and seven categories.

c. Dataset Source and Attribution

Official UCI page: https://archive.ics.uci.edu/dataset/602/dry+bean+dataset

DOI: https://doi.org/10.24432/C50S4B

Citation: Koklu, M. and Ozkan, I. A. (2020), *Multiclass Classification of Dry Beans Using Computer Vision and Machine Learning Techniques*, Computers and Electronics in Agriculture, 174, 105507.

Kaggle manual alternative: https://www.kaggle.com/datasets/zeynepazl/dry-beans/data

d. GitHub Repository Link

`TO BE ADDED AFTER MANUAL GITHUB PUSH`

e. Data Preprocessing

Duplicates are removed after reporting. Features and target are separated, followed by one fixed 80:20 stratified split with `random_state=42`. Median imputation and scaling are fitted only inside pipelines on training data. The Decision Tree skips scaling because it is scale-invariant.

f. Models Used

Logistic Regression, Decision Tree, KNN, Gaussian Naive Bayes, and RBF Support Vector Machine. Logistic Regression is a linear probabilistic baseline; Decision Tree captures nonlinear rules; KNN uses local similarity; GaussianNB is a fast distributional baseline; SVM learns nonlinear margins.

g. Model Evaluation Metrics

Accuracy, macro One-vs-Rest AUC, macro Precision, macro Recall, macro F1, and MCC.

h. Model Comparison Table

Generated as `reports/model_comparison.csv` after running `python run_training.py`. No scores are fabricated in this source package.

i. Model Performance Observations

Generated only after actual execution. Compare leaders in AUC, macro F1, MCC, and precision-recall balance; do not select the winner from accuracy alone.

j. Overall Winning Model

`TO BE POPULATED FROM ACTUAL RESULTS`

k. Streamlit Application Description

The app validates an uploaded labeled test CSV, loads saved Joblib pipelines, calculates six metrics, displays a confusion matrix and classification report, compares all models, and downloads predictions. It never calls `fit()`.

l. Deployment Link

`TO BE ADDED AFTER LIVE DEPLOYMENT AND TESTING`

m. Project Folder Structure

The repository contains configuration, modular `src/` code, notebook, data folders, model outputs, reports, figures, deployment guides, screenshot instructions, and a submission PDF template.

n. Installation Instructions

python -m venv .venv

source .venv/bin/activate # Windows: .venv\Scripts\activate

python -m pip install -r requirements.txt

o. Execution Instructions

python run_training.py

jupyter notebook notebooks/ML_Assignment_2.ipynb

streamlit run app.py

For Kaggle, download the XLSX/CSV manually, place it in `data/raw/`, and switch the clearly marked notebook source block.

p. Conclusion

The architecture provides a reproducible, leakage-safe multiclass workflow and deployment application. Final claims must be based on executed results in BITS Virtual Lab.